NanoCP: Taming Load Imbalance in Data-Expert Parallel Decoding under Variable-Length Workloads

Jiefei Chen*
Fudan University
Shanghai, China
Shanghai AI Laboratory
Shanghai, China

Binbin Lin*
Shanghai Jiao Tong University
Shanghai, China
Shanghai AI Laboratory
Shanghai, China

Jinming Ma*†
Shanghai AI Laboratory
Shanghai, China
jimyma@nano-infra.com

Jiangfei Duan†
The Chinese University of Hong Kong
Hong Kong, China
imjfduan@gmail.com

Haojie Duanmu
Shanghai Jiao Tong University
Shanghai, China
Shanghai AI Laboratory
Shanghai, China

Hao Liu
Shanghai AI Laboratory
Shanghai, China
Harbin Institute of Technology
Harbin, China

Qinxiu Cheng
Shanghai AI Laboratory
Shanghai, China

Xiuhong Li
Peking University
Beijing, China

Zhilin Pei
Shanghai AI Laboratory
Shanghai, China

Hui Wang†
Shanghai AI Laboratory
Shanghai, China
wanghui@pjlab.org.cn

Xingcheng Zhang
Shanghai AI Laboratory
Shanghai, China
SenseTime
Shanghai, China

Dahua Lin
Shanghai AI Laboratory
Shanghai, China
The Chinese University of Hong Kong
Hong Kong, China

arXiv:2605.21100v2 [cs.DC] 21 Oct 2026

Abstract

Modern serving systems for Mixture-of-Experts (MoE) models adopt hybrid data-expert parallelism: expert parallelism (EP) shards experts across GPUs to scale capacity, while data parallelism (DP) replicates attention layers across instances to process independent requests. Existing systems bind each request's attention, MoE communication, and KV cache to a single instance. Because attention latency scales with KV cache size while MoE communication latency scales with batch size, this binding cannot balance both simultaneously, producing EP stragglers; it also fragments KV memory across instances, inflating tail latency under long contexts. While existing context parallelism (CP) mitigates these constraints, existing group-level and uniform CP designs incur prohibitive communication and attention-side overheads.

We present NanoCP, which decouples MoE communication from KV cache placement and achieves dual balance through dynamic context parallelism (DCP). DCP assigns each request a context-parallel degree sized to its KV footprint: long requests distribute attention across multiple instances; short requests remain local.

This dynamic parallelism effectively liquefies the KV cache across DP instances within each node, balancing both the per-instance KV cache occupancy and batch sizes without unnecessary load-balancing costs. To bridge DCP with static execution, NanoCP introduces an ahead-of-time (AOT) graph engine paired with a custom routing-based communication backend. Experimental results show that, under 99% TPOT-SLO attainment, NanoCP achieves 3.27× and 2.11× the maximum sustainable request rate of the best-performing vLLM baseline on the DeepSeek-V3 Issue 1% and Issue 5% workloads, respectively, while reducing P99 TPOT by 1.94× and 2.01× on the same two workloads, respectively.

CCS Concepts: • **Computer systems organization** → **Distributed architectures**; • **Computing methodologies** → **Parallel computing methodologies**.

Keywords: mixture-of-experts, LLM serving, distributed inference, dynamic context parallelism

ACM Reference Format:
Jiefei Chen, Binbin Lin, Jinming Ma, Jiangfei Duan, Haojie Duanmu, Hao Liu, Qinxiu Cheng, Xiuhong Li, Zhilin Pei, Hui Wang, Xingcheng Zhang, and Dahua Lin. 2027. NanoCP: Taming Load Imbalance in Data-Expert Parallel Decoding under Variable-Length

*Equal contribution.
†Corresponding authors.



This work is licensed under a Creative Commons Attribution 4.0 International License.

ASPLOS '27, Heraklion, Greece.

© 2027 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-3012-2/2027/04
https://doi.org/10.1145/3845814.3850079

Workloads. In *Proceedings of the 32nd ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1 (ASPLOS '27), April 11–15, 2027, Heraklion, Greece.* ACM, New York, NY, USA, 18 pages. https://doi.org/10.1145/3845814.3850079

1 Introduction

Mixture-of-experts (MoE) architectures have become the dominant design for scaling large language models (LLMs) beyond the trillion-parameter regime without incurring proportional computation costs [11, 26, 30, 66]. In production, these models serve requests of highly variable length, from short conversational turns to multi-million-token agentic and long-context workloads [14, 18, 28]. Meeting tight token-level latency targets on such workloads is now a central system challenge in LLM inference.

Modern LLM serving systems typically adopt prefill-decode disaggregation [48, 79] together with hybrid *data-expert parallelism* (DP-EP) [11, 31, 71, 77]. In this design, non-expert layers (e.g., attention) are replicated across data-parallel (DP) instances[1], while experts are sharded across those instances through expert parallelism (EP). Each MoE layer therefore incurs two all-to-all communication phases, dispatch and combine, whose latency is determined by the slowest participating rank under collective synchronization. To describe request placement in this architecture, we introduce two terms used throughout the paper. A request's *MoE binding* denotes the DP-instance set responsible for its MoE dispatch and combine operations. In this paper, this set is a singleton. A request's *KV binding* is the set of one or more DP instances that store its KV cache and execute its attention computation. In current request-level serving systems [31, 55, 77], the MoE binding and KV binding are tied to the same DP instance.

Under variable-length workloads, this design creates a fundamental scheduling problem. The two dominant phases in decoding scale differently: attention latency on each instance scales with its resident KV cache size [79], while dispatch and combine latency scale with the per-rank token batch entering the all-to-all [73]. As shown in Figure 1(a), routing for KV cache balance inevitably skews batch sizes, and routing for batch size balance inevitably skews KV cache load. Under lock-step synchronization, both types of imbalance force faster ranks to wait for the slowest one, making end-to-end latency dominated by EP stragglers [6, 22, 45, 60, 61]. This problem is orthogonal to prior work on expert load imbalance, such as EPLB and redundant experts [12].

Tying the MoE binding and the KV binding to the same DP instance also creates a capacity-side problem. As MoE weights grow toward the trillion-parameter regime, model weights consume most of each instance's HBM, leaving KV cache capacity on the same order as a single long-context request. In one representative deployment—a 1T-parameter MoE on 32×H200 with 32DP-32EP—the KV cache capacity of one instance is roughly 1M tokens, already comparable to the context windows of recent models [46, 52]. A single long request can therefore approach the capacity of an entire instance. As a result, even modest fragmentation can prevent the request from being scheduled, although the cluster still has enough aggregate free memory. This creates head-of-line (HoL) blocking and inflates tail latency, as shown in Figure 1(a).

A related line of work is context parallelism (CP) [2, 4, 5, 17, 25, 27, 32, 36, 37, 62–64, 67], which expands a request's KV binding across multiple DP instances. However, existing CP systems are mainly designed for dense models or uniformly long-context serving, rather than variable-length MoE decoding. LoongServe [63] elastically adjusts CP at the group level. Applied to DP-EP decoding, larger CP degrees for long-request groups reduce attention computation imbalance but cause batch size imbalance across DP instances, leading to EP stragglers. Helix [4] instead applies the same CP degree to every request, as illustrated in Figure 1(b). It keeps one MoE binding per request but uniformly expands every request's KV binding to the same fixed-size group. This forces short requests to incur unnecessary cross-instance communication and increases the effective attention batch size on each CP instance, thereby reducing attention-kernel efficiency. Neither LoongServe nor Helix independently selects each request's MoE binding and KV binding within the same decoding step, preventing them from jointly balancing attention computation and MoE communication with low CP overhead.

To address this problem, we present NanoCP, which introduces dynamic context parallelism (DCP) at per-request granularity to decouple a request's MoE binding from its KV binding and schedule them independently. NanoCP chooses each request's MoE binding to balance batch sizes across DP instances, while choosing each request's KV binding by trading off better attention computation balance against additional communication overhead. Under DCP, a request's CP degree is the number of DP instances in its KV binding. NanoCP therefore assigns larger CP degrees to long requests to better balance attention computation, while assigning smaller CP degrees to short requests to avoid unnecessary communication. As illustrated in Figure 1(c), this design achieves both batch size balance and KV cache balance while avoiding the attention-kernel efficiency loss and high CP communication overhead of uniform CP.

In the control plane, NanoCP uses a centralized scheduler with a global view of load across instances, since local schedulers cannot jointly balance these dimensions or support token-level KV cache placement. Based on this global view, the scheduler determines, for each request, where MoE communication executes, what CP degree it uses, and how its KV cache is distributed across instances. However, executing

[1] A DP instance is the set of GPUs that together hold one replica of the non-expert layers and one shard of the experts.

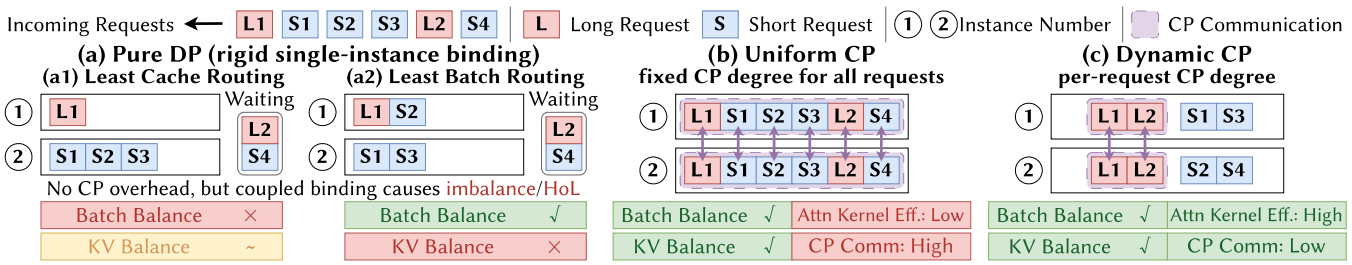


Figure 1. Existing load balancing strategies: (a1) least cache routes each incoming request to the instance with the largest number of free KV blocks, (a2) least batch routes it to the instance with the smallest running batch size, and (b) uniform CP achieves dual balance but incurs low attention-kernel efficiency and high CP communication; (c) NanoCP.

this per-request flexibility creates highly asymmetric cross-instance traffic that conflicts with the static-shape assumptions of CUDA Graph capture and DeepEP's low-latency decode kernels [73]. To support this efficiently in the data plane, NanoCP pairs an ahead-of-time (AOT) graph manager, which pre-compiles a bounded family of static-shape execution graphs, with a routing-based communication backend that issues asymmetric transfers outside the collective path.

In summary, this paper makes the following contributions:

1. We identify rigid request-to-instance binding as a fundamental source of both EP stragglers and memory fragmentation in hybrid DP-EP MoE serving, and show why existing CP designs are inefficient under variable-length workloads.
2. We propose NanoCP, a *dynamic context parallelism* (DCP) system that decouples each request's MoE binding from its KV binding within the same decoding step and supports per-request CP degrees under static-shape GPU runtimes.
3. We implement NanoCP and show that under 99% TPOT-SLO attainment, it achieves 3.27× and 2.11× the maximum sustainable request rate of the best-performing baseline on the DeepSeek-V3 Issue 1% and Issue 5% workloads, respectively [4, 31]. At the request rate where the best-performing vLLM baseline reaches 100 ms mean TPOT, NanoCP achieves 1.87×–2.10× mean-TPOT speedups on the Issue 1% and Issue 5% workloads, evaluated separately with DeepSeek-V3 and Kimi-K2.

We have open-sourced NanoCP. [2]

2 Background and Motivation

2.1 Variable-Length Workloads

Modern LLM serving systems must handle requests with highly variable sequence lengths, ranging from short conversational prompts [65, 74, 76] to much longer multimodal and agentic workloads [14, 28, 49]. As shown by the OpenRouter statistics in Table 1 [3, 47], production traffic is highly skewed: most requests are short, but a non-negligible fraction (1.67%) exceeds 100k tokens. Following Medha [2], we construct evaluation traces by mixing ShareGPT-4o [9] for short requests with long-context software-engineering traces derived from GitHub issues for long requests. We use long-request ratios of 1% and 5% to represent both typical production workloads and heavier long-context stress cases.

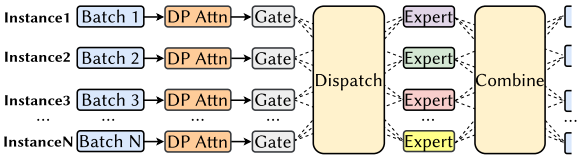


Figure 2. Data-expert parallel architecture.

2.2 Data-Expert Parallel MoE Serving

As shown in Figure 2, modern MoE decoding typically adopts data-expert parallelism (DP-EP) rather than tensor parallelism (TP) [53, 70, 81]. Head-wise TP is poorly suited to Multi-Head Latent Attention (MLA) [11], as it can replicate the KV cache and erode MLA's memory-efficiency advantage [11, 57]. Meanwhile, sharding small experts produces fragmented GEMMs that fail to saturate GPU compute units [81]. DP-EP therefore replicates attention layers across DP instances while sharding experts through EP, enabling request-level parallelism and leveraging aggregate memory bandwidth [70, 73, 81]. Each MoE layer performs one dispatch and one combine all-to-all.

Different scaling factors in data-expert parallelism. A fundamental bottleneck in globally synchronized DP-EP execution is *instance-level imbalance*: attention latency scales with the total KV cache size (Figure 3(a)), whereas MoE communication latency scales with batch size (Figure 3(b)). Figure 3(a) also shows that larger effective batches can reduce attention-kernel efficiency at the same KV-token count; we discuss this effect in Section 2.3.2 when analyzing uniform CP.

[2] https://github.com/nano-infra/NanoEngine/tree/decentral-aug

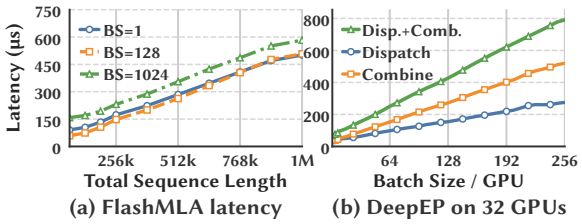


Figure 3. Microbenchmarks of (a) attention latency versus the total number of cached tokens per GPU under different batch sizes, where sequence length = total tokens / batch size and (b) DeepEP communication latency versus batch size.

Table 1. Request-length distributions (%).

Dataset	< 1k	1k–10k	10k–100k	100k–500k	500k–1M
OpenRouter[1]	31.82	50.08	16.42	1.67	
ShareGPT-4o	85.7	10.7	3.5	-	-
GitHub Issues	-	-	-	65.06	34.94

[1] Sourced from https://openrouter.ai/rankings, representing the call count records for Gemini 3 Flash Preview on Feb. 23.

Under variable-length workloads, KV cache load and batch size can diverge, making them difficult to balance simultaneously. Lock-step synchronization then causes the slowest instance to stall the entire DP-EP group. Prior work primarily addresses *expert-level imbalance* through expert placement optimization [11, 20, 34, 70] or training-time load balancing [15]. However, even perfectly balanced experts do not eliminate this instance-level bottleneck.

2.3 Existing Load Balancing Strategies

2.3.1 Request-Level Load Balancing. A common baseline for hybrid DP-EP serving is *request-level load balancing*, which assigns each request to a single DP instance. As illustrated by the DP case in Figure 7(a), this design ties the request's MoE binding and KV binding to that same DP instance. This coupling is restrictive because attention scales with KV cache size, whereas MoE communication scales with batch size. As a result, this request-level scheduling can balance only one dimension.

LeastCache strategy and communication imbalance. This policy routes each incoming request to the DP instance with the smallest total KV cache size to balance attention computation. We implement this policy in vLLM for comparison. However, by ignoring batch size, it can create severe batch size imbalance, leading to communication imbalance and stragglers. As illustrated in Figure 4(a), even when attention loads are balanced, DP instance 0 may handle only 2 requests while DP instance 1 handles 98. DP instance 0 therefore spends much longer in dispatch receive (DR), delaying its combine send (CS). As a result, DP instance 1, despite

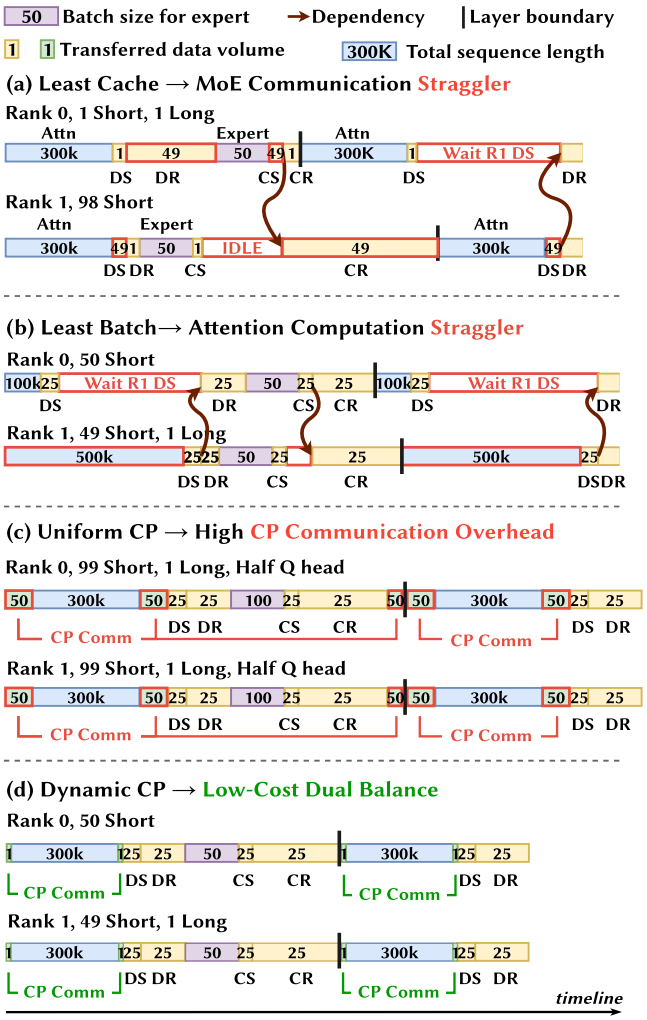


Figure 4. Per-layer timelines of four MoE decode strategies. (DS/DR: Dispatch Send/Receive; CS/CR: Combine Send/Receive).

finishing its MLP and combine send quickly, is blocked before combine receive (CR) and remains idle. This straggler arises from batch size imbalance together with the sequential dependency between dispatch and combine. Using 32 DeepSeek-V3 instances driven by the Issue 1% trace in vLLM, with DP degree 32 and a constant arrival rate of 30 requests/s, Figure 5(a) shows that MoE dispatch and combine latencies vary significantly across instances: most instances fall between 100 and 500 μs, while the straggler reaches 1020.6 μs. The mean is 354.7 μs, indicating that perfect load balance would reduce the maximum latency by 65.2%.

LeastBatch strategy and attention computation imbalance. This policy routes each incoming request to the DP instance with the smallest batch size to balance MoE communication. vLLM's default scheduler follows this policy. However, because it ignores request-length heterogeneity,

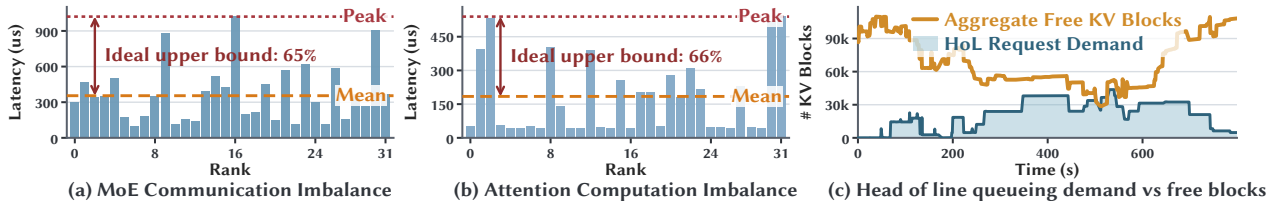


Figure 5. Limitations of existing request-level load balancing strategies.

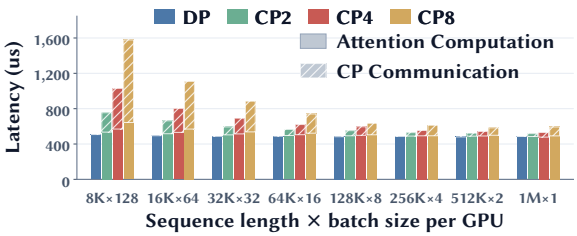


Figure 6. Per-layer attention latency breakdown of uniform CP under varying sequence lengths and batch sizes (total ~1M tokens per GPU).

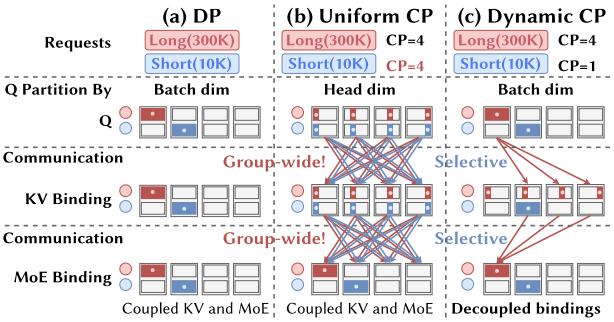


Figure 7. KV and MoE binding comparison.

it can create severe KV cache imbalance and stragglers. As shown in Figure 4(b), even when batch sizes are balanced, attention loads can still differ substantially (e.g., 100K vs. 500K tokens). DP instance 0 finishes its lighter attention phase and dispatch send (DS) quickly, but cannot proceed because DP instance 1 is still executing a heavier attention computation. This delays DP instance 1's dispatch send and leaves DP instance 0 idle. Thus, stragglers arise from KV cache imbalance together with the dependency between attention and dispatch. Figure 5(b) shows that attention latency also varies significantly under this policy: most instances remain around 40–55 μs, while the straggler reaches 539.5 μs. The mean is 184.3 μs, indicating that balancing attention computation would reduce the maximum latency by 65.8%.

Head-of-line blocking. Request-level routing also leads to severe head-of-line (HoL) blocking. Since each request must be placed on a single DP instance, a long-context request with a large KV cache cannot enter execution until one instance has enough local memory. Even when the cluster has sufficient aggregate free memory, the request can remain stuck at the head of the queue and block subsequent requests. Figure 5(c) shows this effect: there is a clear gap between the high total free memory and the blocked request at the front of the queue. Because the KV cache of a request cannot be distributed across instances, aggregate free memory cannot be effectively utilized, leading to higher tail latency.

2.3.2 Uniform CP. Helix [4] is a representative uniform CP design that improves over request-level scheduling by allowing multiple GPUs to jointly execute each request's attention, thereby distributing its KV cache and reducing attention stragglers. As illustrated in Figure 7(b), uniform CP keeps each request's MoE binding on one DP instance while expanding its KV binding to a fixed CP group.

However, as shown in Figure 4(c), uniform CP also forces short requests to participate in cross-instance attention execution. Partitioning their KV cache and query activations across the group requires routing queries, returning partial attention results, and merging them into the final output, introducing unnecessary communication overhead.

Figure 6 quantifies this overhead under different sequence-length and batch-size combinations. Compared with DP, all CP configurations (CP2, CP4, and CP8) incur substantially higher communication costs. The overhead is especially pronounced for short sequences with large batches (e.g., 8K×128), where CP communication accounts for a large fraction of the total attention latency.

Uniform CP can also reduce attention-kernel efficiency by increasing the effective batch observed by each CP instance. As shown in Figure 3(a), at a fixed total KV-token count, a larger effective batch divides the workload among more, shorter sequences. Because FlashMLA uses a persistent tile scheduler, processing more sequences incurs additional sequence-specific handling and tail-tile inefficiency, whose costs are amortized over fewer KV tokens. Here, BS denotes the effective attention-kernel batch after CP redistribution, rather than the number of independent user requests on one DP instance. NanoCP avoids these inefficiencies by keeping short requests local and assigning larger CP degrees only to long requests.

2.4 DCP with Dual-Balanced Scheduling

Figure 7 illustrates how the three designs differ in request binding. In DP, each request's KV binding and MoE binding are tied to the same instance. Helix-style CP expands the KV binding of every request to a fixed CP group, so short and long requests both pay group-wide attention communication. NanoCP keeps one MoE binding per request but chooses the KV binding size dynamically, allowing long requests to use CP while keeping short requests local.

To jointly balance attention computation and MoE communication with low overhead, we propose dual-balanced scheduling with dynamic context parallelism (DCP). The key idea is to decouple a request's MoE binding from its KV binding, so that MoE communication and KV cache placement can be scheduled independently within the same decoding step. As shown in Figure 7, NanoCP selects each request's singleton MoE binding to balance per-instance batch sizes for dispatch and combine, and independently selects its KV binding to balance KV cache and attention computation subject to cross-instance communication cost.

DCP enables this design efficiently in two ways. First, NanoCP distributes the KV cache and attention computation of long requests across multiple instances only when doing so improves load balance, while keeping short requests local to avoid unnecessary communication. As illustrated in Figure 4(d), NanoCP makes Instance 0 and Instance 1 each hold 300K tokens of KV cache and process a batch size of 50, while incurring much lower communication overhead than Helix-style CP, leading to the shortest execution time in this example. Second, by partitioning a long request's KV cache within its selected KV binding, NanoCP avoids requiring the request to fit into the free memory of a single instance, thereby utilizing fragmented free memory across instances and alleviating head-of-line blocking.

3 NanoCP Overview

Figure 8 illustrates the NanoCP architecture. The control plane (Section 4) maintains a *global state manager* (Section 4.1) that pools all requests and GPU resources into a unified view. Based on this view, the *dual-balanced scheduling strategy* (Section 4.2) generates per-iteration execution plans, determining each request's *MoE binding* and *KV binding*, and dispatches per-instance metadata to the data plane (Section 5). Since decoding relies on CUDA Graph to reduce CPU launch and synchronization overhead, per-request DCP's varying routing patterns conflict with its static-shape assumptions. NanoCP therefore combines an *AOT graph manager* (Section 5.2) with a *routing-based communication backend* (Section 5.3) to support dynamic routing efficiently.

Specifically, a request's decode lifecycle flows through this architecture: After completing external prefill, a request enters a global waiting queue (❶ ENQUEUE). When admitted into decode, the scheduler determines its MoE binding, KV

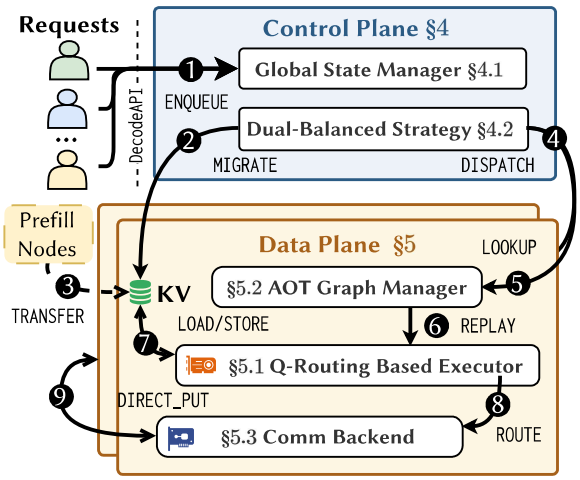


Figure 8. NanoCP system overview.

binding, and KV-token split. Long requests may span multiple instances to distribute attention load and reduce KV cache imbalance, whereas short requests typically remain local to avoid unnecessary communication. The control plane then triggers KV cache migration to the selected instances (❷ MIGRATE), while the data plane performs the transfer (❸ TRANSFER), and lowers the scheduling decision into per-instance routing metadata (❹ DISPATCH). Each instance uses this metadata to look up (❺ LOOKUP) and replay (❻ REPLAY) the matching prebuilt graph.

During execution, the MoE binding sends the query tensor to the instances in the KV binding via the routing backend (❽ ROUTE). Each instance performs attention over its local KV shard (❼ LOAD/STORE) and returns a partial result (❾ DIRECT_PUT), which the MoE binding merges before proceeding to MoE execution.

4 NanoCP Control Plane

4.1 Global State Management

Unlike mainstream LLM serving systems [31, 36, 40, 55], which tightly couple the scheduler and executor within each data parallel (DP) instance, NanoCP maintains a single global scheduler to coordinate all DP instances. This design is driven by two requirements. First, hybrid DP-EP execution requires strict *per-iteration* load balancing, since the MoE dispatch and combine phases proceed in lock-step across DP instances and any imbalance immediately creates stragglers. Second, DCP requires fine-grained resource allocation across instances, including token-level partitioning of a request's KV cache to reduce execution imbalance and memory fragmentation, which is difficult under coupled architectures where each scheduler manages only local resources.

4.2 Dual-Balanced Scheduling

NanoCP realizes dual-balanced scheduling as a request-centric, length-aware scheduling algorithm. For each request,

Algorithm 1 Dual-Balanced Scheduling Strategy

Require: $\mathcal{N}$, Q_{new}, Q_{act}, $\mathbf{K}$, $K_{\max}$, $\textsc{Bucket}(\cdot)$
Ensure: Updated MoE assignments m_r for active requests; $(\mathcal{P}_r, m_r, Split_r)$ for each admitted $r \in Q_{new}$
Rebalance MoE Communication Assignments for Active Requests
1: $B \leftarrow \mathbf{0}$
2: **for** each $r \in \textsc{SortByParticipantCountAsc}(Q_{act})$ **do**
3: $m_r \leftarrow \arg\min_{s \in \mathcal{P}_r} B_s$
4: $B_{m_r} \leftarrow B_{m_r} + 1$
5: **end for**
Node Selection and CP Degree
6: **for** each $r \in Q_{new}$ **do**
7: $n^* \leftarrow \arg\min_{n \in \mathcal{N}} B^{(n)}$
8: $k_r \leftarrow \min(\textsc{Bucket}(\ell_r), |\mathcal{S}_{n^*}|)$
Intra-Node Placement
9: $m_r \leftarrow \arg\min_{s \in \mathcal{S}_{n^*}} B_s$
10: $C_r \leftarrow \textsc{SelectSmallestKV}(\mathcal{S}_{n^*} \setminus \{m_r\}, k_r - 1)$
11: $\mathcal{P}_r \leftarrow \{m_r\} \cup C_r$
12: $Split_r \leftarrow \textsc{WaterFill}(\mathcal{P}_r, \ell_r, \mathbf{K})$
13: **if** $\textsc{CanAllocate}(\mathcal{P}_r, Split_r, \mathbf{K}, K_{\max})$ **then**
14: Commit $(n^*, \mathcal{P}_r, m_r, Split_r)$ for request r
15: Update local KV loads K_s and MoE comm. load B_{m_r}
16: **else**
17: Keep r in waiting queue for the next scheduling iteration
18: **end if**
19: **end for**

it determines two decoupled bindings, with one placing its MoE communication on a single instance and the other assigning its KV cache to a set of instances for attention computation. These decisions jointly balance MoE communication load and attention computation load across instances. Let $\mathcal{N}$ denote the set of nodes and $\mathcal{S}_n$ the CP instances on node n. For request r, ℓ_r is its sequence length, $\mathcal{P}_r$ is the selected instance set for attention computation, m_r is the instance assigned to execute its MoE communication, and $Split_r$ maps each instance in $\mathcal{P}_r$ to its assigned tokens. For instance s, K_s denotes its KV cache load, and $\mathbf{K} = (K_s)_s$ denotes the current KV-load state; B_s denotes the number of requests whose MoE communication is assigned to instance s in the current decoding iteration; $B^{(n)} = \sum_{s \in \mathcal{S}_n} B_s$, and $K_{\max}$ is the per-instance KV cache capacity. We use Q_{act} to denote active requests with their existing KV bindings $\mathcal{P}_r$, and Q_{new} to denote requests considered for admission.

4.2.1 Rebalancing MoE Communication Assignments for Active Requests.

As requests complete, MoE communication assignments can become imbalanced. Since an active request can be reassigned within its existing KV binding $\mathcal{P}_r$ without data migration, the scheduler processes requests in ascending order of $|\mathcal{P}_r|$, prioritizing those with fewer reassignment choices, and assigns each request to the instance in $\mathcal{P}_r$ with the fewest assigned requests (lines 2–3).

4.2.2 Node Selection and CP Degree.

For each new request, the scheduler selects the node whose instances own the fewest total requests (line 7). It then determines the CP degree k_r using a length-bucket function $\textsc{Bucket}(\ell_r)$ (line 8). The key insight is that the optimal CP degree increases with request length: longer requests have larger KV cache footprints and thus lead to KV cache imbalance, so distributing them across more instances improves KV cache balance and reduces head-of-line blocking; shorter requests benefit less from CP and are more sensitive to its communication overhead. Based on this insight, $\textsc{Bucket}(\cdot)$ is derived from offline profiling. We sweep sequence lengths and candidate CP degrees, measure distributed attention latency under DCP, including both attention computation and communication, and select the CP degree that minimizes this latency for each length range. At runtime, the scheduler determines k_r by looking up $\textsc{Bucket}(\ell_r)$. This mapping depends on the model and system configuration, rather than the request-length distribution.

4.2.3 Intra-Node Placement and KV Partitioning.

Our implementation keeps each KV binding within one node to avoid critical-path cross-node traffic, while the abstraction supports cross-node placement.

Within the selected node, the scheduler first chooses the instance that will execute the request's MoE communication (line 9). It then selects the remaining participating instances from those with the smallest KV cache load K_s (line 10), forming the final set $\mathcal{P}_r$ for attention computation and KV placement (line 11). Given this participant set, the scheduler computes a token split $Split_r$ using WaterFill (line 12), which distributes tokens to lower-loaded instances first. Specifically, WaterFill assigns tokens such that $\sum_{s \in \mathcal{P}_r} Split_r[s] = \ell_r$, while minimizing the peak post-allocation KV load $\max_{s \in \mathcal{P}_r}(K_s + Split_r[s])$. The scheduler commits the placement only if CanAllocate confirms feasibility (line 13); otherwise, the request remains in the waiting queue for the next scheduling iteration (line 17).

4.2.4 Routing Table Generation.

Once the scheduler finalizes placement decisions and token splits, it lowers them into per-instance Q-routing and Res/LSE routing tables, which decouple dynamic scheduling from data-plane execution. Section 5.1 details how the data plane consumes these tables to drive Q-routing execution (Figure 9).

5 NanoCP Data Plane

The DCP execution engine orchestrates the dual-balanced execution plan generated by the control plane.

5.1 Decoupled Data-Path Design

Given the execution plan generated by the control plane, NanoCP uses a Q-routing data path between the *MoE binding* and *KV binding*: the singleton MoE binding performs

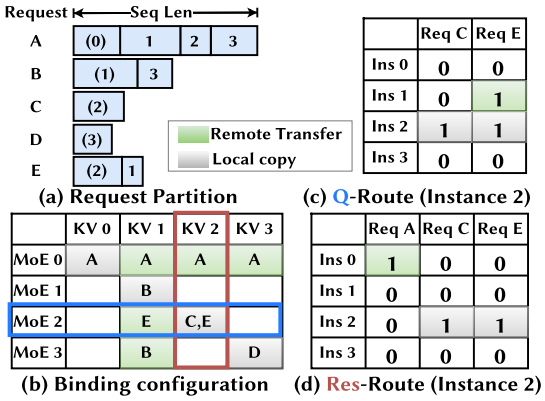


Figure 9. Routing table derivation. (a) Request partition with MoE binding assignment (parenthesized). (b) Binding configuration. (c)–(d) Per-instance Q-Route and Res-Route tables for Instance 2.

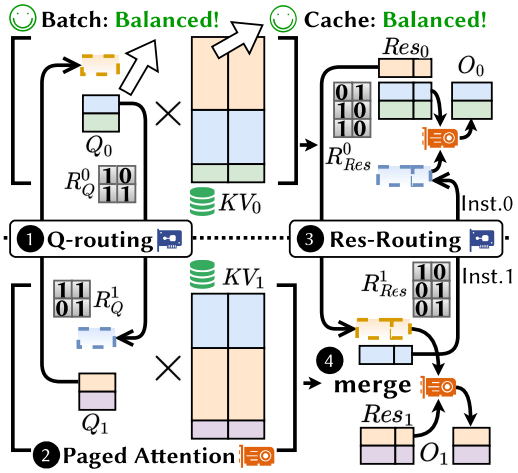


Figure 10. Execution process of NanoCP DCP.

MoE dispatch and combine, routes the lightweight query (Q) tensor to the KV-binding instances for distributed attention, and receives partial results for merging. This avoids migrating KV cache between the MoE and KV bindings during decoding. While similar Q-routing ideas have been explored in recent systems [36, 63, 67], NanoCP makes dynamic routing compatible with static GPU runtimes through an ahead-of-time (AOT) context-driven graph engine (Section 5.2) and a routing-based communication backend (Section 5.3).

Before each decoding iteration, the control plane derives per-instance routing tables from the KV cache distribution. For a given instance s, let M and N denote its local MoE and attention batch sizes, respectively: M is the number of requests whose MoE binding includes s, and N is the number whose KV binding includes s. Figure 9 illustrates this conversion for five requests across four instances. For example, C and E have Instance 2 as their MoE binding (M=2), while A, C, and E include Instance 2 in their KV bindings (N=3). Let W_{cp} denote the number of instances in the DCP routing domain. For each instance, the control plane derives two binary tables: *Q-Route* ($W_{cp} \times M$), indicating the KV-binding instances to which each locally generated query is routed, and *Res-Route* ($W_{cp} \times N$), indicating the MoE-binding instance to which each local partial result is returned. Transfers are issued only for entries marked 1.

Figure 10 illustrates the execution workflow. In **Phase 1** (*Projection & Q-Routing*), the MoE binding computes Q and routes it to the KV binding via the Q-Route table (❶). Each KV-binding instance then computes a partial attention output (P) and its log-sum-exp (LSE) scaling factor over its local KV partition (**Phase 2**, ❷). The partial results are returned via the Res-Route table (**Phase 3**, ❸) and merged at the MoE binding using LSE merging [10] (**Phase 4**, ❹) before proceeding to the MoE layer.

Algorithm 2 AOT Graph Capture and Replay

Require: Execution-shape buckets $\mathcal{S}$ covering all supported (M, N)

— **Initialization: Graph Capture** —
$\mathcal{B} \leftarrow$ AllocateSharedPool($\mathcal{S}$)
for each bucket $b = (\hat{M}, \hat{N}) \in \mathcal{S}$ **do**
  $\mathcal{C}[b] \leftarrow$ CreateView($\mathcal{B}, b$)
  $\mathcal{G}[b] \leftarrow$ CaptureGraph($\mathcal{C}[b]$)
end for
— **Runtime: Graph Replay** —
$b \leftarrow$ SmallestFit($(M, N), \mathcal{S}$)
AsyncUpdate($\mathcal{C}[b]$, inputs and routing metadata)
Replay($\mathcal{G}[b]$)

5.2 Bridging Dynamic Routing and Static Graphs

A fundamental challenge in implementing DCP is that per-request CP degrees and KV bindings vary across iterations, changing both Q and Res routing and the set of requests executing attention on each instance. Thus, the per-instance execution shape (M, N) is dynamic. A CUDA Graph, however, replays a predetermined set of GPU operations with fixed launch shapes and buffer layouts. CPU-side orchestration would introduce significant synchronization overhead on the critical path [7, 19, 50, 59, 75]. Alternatively, for each candidate M, capturing a graph that reserves the worst-case attention batch size $N = M \cdot W_{cp}$ requires padding smaller executions, inflating the effective attention batch and degrading attention-kernel efficiency, as shown in Section 2.3.2.

NanoCP addresses this mismatch with an ahead-of-time (AOT) graph engine (Algorithm 2). We define a bounded set $\mathcal{S}$ of two-dimensional execution-shape buckets $b = (\hat{M}, \hat{N})$, such that each supported runtime shape (M, N) can be mapped to a bucket with $\hat{M} \geq M$ and $\hat{N} \geq N$. During initialization, the engine pre-allocates shared buffers large enough for the largest bucket. For each bucket, it creates tensor views of the required sizes with fixed memory addresses

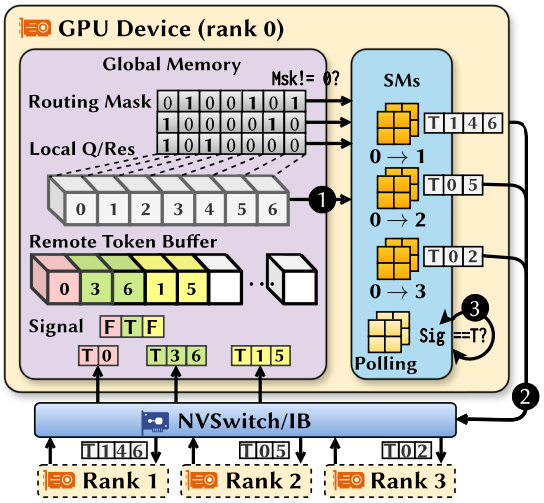


Figure 11. The microarchitecture of the routing-based communication backend.

from these shared buffers and captures a complete DCP execution graph over these views. At runtime, the engine selects the smallest bucket that accommodates the current (M, N), asynchronously updates its metadata, and replays the corresponding graph. This design allows all bucket-specific graphs to reuse the same underlying buffers while avoiding per-operator CPU orchestration and limiting padding to the size of the selected bucket.

5.3 Routing-Based Communication Backend

To efficiently execute dynamic routing between decoupled KV binding and MoE binding within CUDA graphs, existing communication libraries do not match DCP's communication pattern for two reasons. First, general collective libraries [8, 24, 44, 51] are optimized for dense exchanges with fixed layouts. Supporting DCP's sparse and asymmetric transfers therefore requires excessive zero-padding, wasting bandwidth and increasing latency. Second, specialized MoE libraries [21, 35, 38, 41, 54, 73] are designed around fixed-width Top-K token-to-expert routing. In contrast, DCP allows requests in the same batch to use different CP degrees, creating variable per-request fan-out/fan-in. Mapping this pattern to fixed-width Top-K requires padding to the maximum CP degree or multiple communication invocations. We therefore design a routing-based backend tailored to DCP's sparse and variable-width routing.

Unlike traditional MoE routing, whose destinations depend on runtime gating outputs, DCP's communication topology is determined by the control plane before each iteration. The backend therefore uses the routing table to directly steer data transfers, avoiding translation into a fixed-width Top-K representation and sending each data chunk directly to its target rank.

As illustrated in Figure 11, the runtime interprets the routing table as a sparse 2D mask matrix. For each data chunk, the sender reads the corresponding mask entries from global memory (❶) to identify its target ranks, then transfers the payload directly into their preallocated receive buffers via NVLink for intra-node routing or InfiniBand (IB) for inter-node routing (❷). In parallel, the receiver polls a local arrival flag (❸).

6 Evaluation

6.1 Experimental Setup

We implement NanoCP in ~10K lines of code atop FlashMLA [33], Triton-distributed [78], DeepEP [73], DeepGEMM [72], nano-vllm [69], and Ray [43]. The control plane is written in C++ and sends scheduling metadata to the data plane via RDMA to minimize overhead. NanoCP also supports multi-step execution [39], reusing one scheduling decision across multiple decoding iterations.

We evaluate NanoCP on a cluster of NVIDIA H200 GPUs. Each node has eight GPUs connected by a fully-connected NVLink fabric (900 GB/s bidirectional) and eight 50 GB/s RDMA NICs for inter-node communication. We use DeepSeek-V3 [11] and Kimi-K2 [29] as model backbones.

Workloads. We use ShareGPT-4o [9] as the short-context dataset and GitHub Issues [2] as the long-context dataset. We synthesize mixed workloads by combining them at 1% and 5% long-context ratios, and also evaluate pure short- and long-context baselines.

Baselines. Both NanoCP and vLLM [31] use equivalent kernels (FlashMLA for attention, DeepEP [73] for MoE all-to-all). All configurations use wide EP for expert layers. For attention, we compare:

- **32DP:** Standard data-parallel attention across 32 GPUs with vLLM's default policy (LeastBatch) and an additional LeastCache policy.
- **DP-CP Hybrid:** 16DP2CP, 8DP4CP, and 4DP8CP, using vLLM's native uniform CP with head-dimension partitioning, following Helix's uniform decode-CP data path [4]. We do not include HOP-B described in Helix, as no directly usable open-source implementation is available.

Metrics. Our primary metric is time-per-output-token (TPOT) with a 50 ms SLO target [23, 54]. We report the maximum request rate at which each system sustains ≥99% SLO attainment, i.e., at least 99% of requests meet this TPOT target.

6.2 Micro Benchmark

We compare NanoCP against four vLLM baselines (vLLM 32DP, vLLM 16DP2CP, vLLM 8DP4CP, vLLM 4DP8CP) on 4 nodes, placing 64 short requests (2048 tokens) per GPU and 1, 3, 5, or 7 long requests (512K tokens) per node. For each setting, we report the latency breakdown of the slowest instance, since EP makes per-layer latency nearly identical

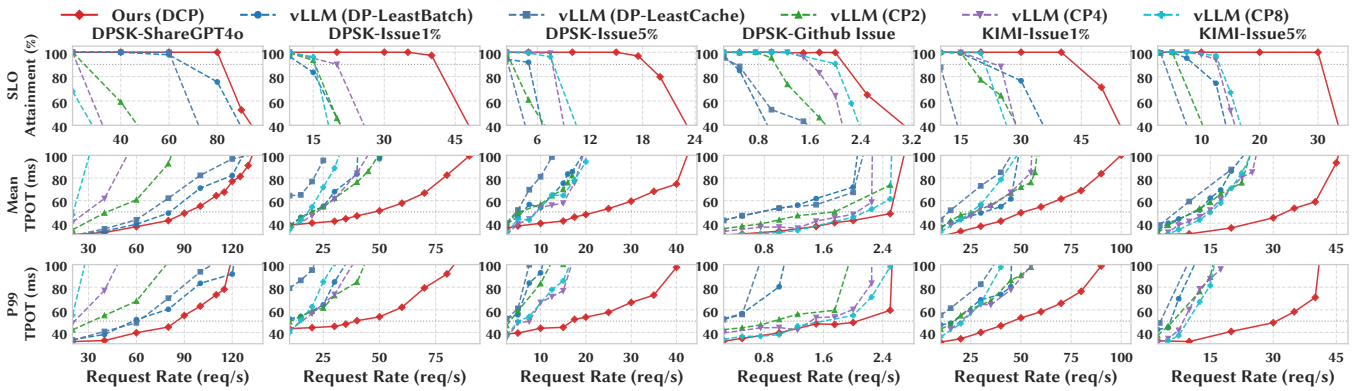


Figure 12. End-to-end generation performance on the H200 cluster.

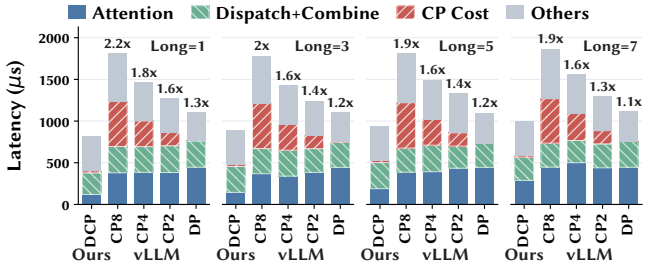


Figure 13. Per-layer latency breakdown with varying numbers of long requests per node.

across instances. Per-layer latency is decomposed into attention computation, MoE all-to-all, CP communication, and other costs.

Figure 13 shows that NanoCP achieves the lowest overall latency across all four settings. Its CP communication overhead is much smaller than vLLM 4DP8CP's, because it enables CP only for requests that would otherwise create significant attention imbalance. NanoCP also achieves the smallest maximum attention latency: compared with vLLM 32DP, it distributes long-request KV cache across instances rather than confining each to a single DP instance; compared with vLLM 4DP8CP, it avoids the inflated attention batch of uniform CP, where each CP instance executes attention over the full CP-group batch, hurting kernel efficiency. vLLM 16DP2CP and vLLM 8DP4CP suffer from both attention stragglers and reduced kernel efficiency.

The benefit of distributing long-request KV cache diminishes as the number of long requests increases per node: when they are few, this removes dominant attention stragglers; as their number grows, the room for further KV cache balancing shrinks.

6.3 End-to-End Performance

Figure 12 reports the end-to-end serving performance. Under mixed workloads, vLLM 32DP suffers from imbalances in both attention computation and MoE dispatch/combine communication, leading to stragglers. Uniform CP baselines partially alleviate this problem, but still underperform NanoCP: larger CP groups better balance long requests but incur higher communication and enlarge the effective attention batch, reducing attention-kernel efficiency, whereas smaller CP groups reduce communication but leave substantial attention imbalance. By assigning CP degrees dynamically per request, NanoCP mitigates these imbalances while avoiding the uniform overhead of fixed-degree CP. On DeepSeek-V3, under 99% SLO attainment, NanoCP supports 3.27× and 2.11× the maximum sustainable request rate of the best-performing vLLM baseline on Issue 1% and Issue 5%, respectively.

Under the pure short-context workload (ShareGPT-4o), NanoCP achieves slightly lower latency than vLLM 32DP, showing that selective DCP introduces negligible overhead when context parallelism is unnecessary. Under the pure long-context workload (GitHub Issues), NanoCP performs similarly to 4DP8CP. At such low request rates, CP8 adds little overhead and already reduces stragglers effectively.

At the request rates where the best-performing baselines reach a P99 TPOT of 100 ms, NanoCP reduces P99 TPOT by factors of 1.94× and 2.01× on Issue 1% and Issue 5%, respectively. By distributing a request's KV cache across multiple DP instances, NanoCP mitigates head-of-line blocking and prevents long-context requests from blocking subsequent ones.

The gains also generalize to Kimi-K2. Under 99% SLO attainment, NanoCP sustains 1.88× and 2.20× the maximum request rates of the best-performing vLLM baselines on Issue 1% and Issue 5%, respectively. At the request rates where the best-performing baselines reach 100 ms mean and P99 TPOT, respectively, NanoCP achieves 1.87×–2.07× mean-TPOT speedups and 1.79×–2.12× P99-TPOT speedups. These results show that DCP's benefits are not specific to DeepSeek-V3.

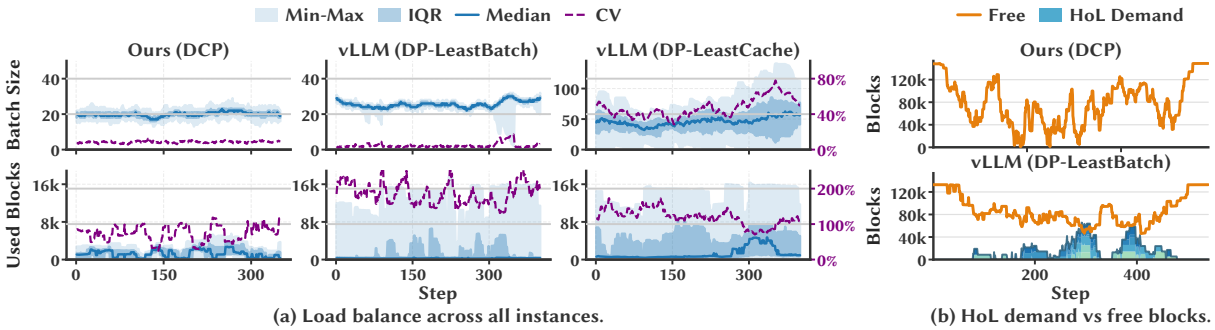


Figure 14. (a) Batch size and KV cache load balance across 32 instances under low load. (b) Head-of-line blocking under high load: free memory blocks vs. head-of-queue demand per node.

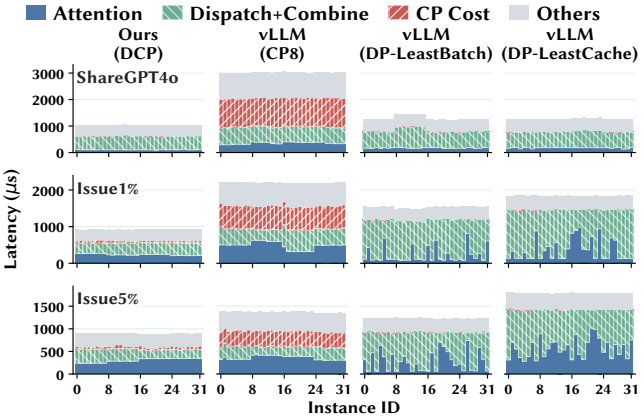


Figure 15. Latency breakdown of different routing and parallel strategies across 32 GPUs

6.4 Load Balance Analysis

NanoCP achieves better load balance and less HoL blocking than vLLM under both low and high load. We quantify cross-instance imbalance using the coefficient of variation (CV), defined as the standard deviation divided by the mean; lower CV indicates better load balance.

Low load. Figure 14 shows that NanoCP keeps batch-size CV close to LeastBatch (7.34% vs. 3.24%) while substantially reducing KV cache CV (51.15% vs. 156.77%). Compared with LeastCache, NanoCP reduces both batch-size CV (7.34% vs. 59.90%) and KV cache CV (51.15% vs. 87.30%).

High load. Figure 14 compares NanoCP against vLLM 32DP with its default scheduling policy. For each 8-GPU node, we compare the aggregate free memory blocks with the demand of the head-of-line request. In vLLM, substantial free memory often coexists with a blocked head-of-line request due to fragmentation. In NanoCP, requests queue mainly when aggregate free memory is nearly exhausted, showing that distributing a request's KV cache across instances mitigates HoL blocking by utilizing fragmented memory.

6.5 Latency Breakdown

We decompose per-layer execution time into attention, MoE dispatch and combine, CP communication, and others across four configurations: NanoCP, vLLM 4DP8CP, vLLM 32DP-LeastBatch, and vLLM 32DP-LeastCache. We evaluate ShareGPT-4o and its 1%/5% GitHub Issues mixtures by replaying requests extracted from serving logs across 32 instances (Figure 15).

Both vLLM 32DP baselines exhibit pronounced attention imbalance. Under ShareGPT-4o + 1% GitHub Issues, vLLM 32DP LeastBatch exhibits a 7.26× max-to-median latency gap (811.3μs vs. 111.8μs), while vLLM 32DP LeastCache still peaks at 986.065μs. By distributing long-context KV cache across instances, NanoCP limits the maximum attention latency to 267.744μs and reduces the max-to-median gap by over 84%.

MoE dispatch+combine shows a similar straggler pattern due to attention imbalance and batch size skew: vLLM 32DP LeastBatch and vLLM 32DP LeastCache reach maximum latencies of 1132.9 μs and 1342.7 μs, respectively. By jointly balancing attention and MoE phases, NanoCP lowers the maximum dispatch-and-combine latency to 350.9 μs.

By enabling DCP only for requests that need it, NanoCP incurs much lower CP communication overhead than vLLM 4DP8CP: 60.4 μs on average versus 629.8 μs, a 90.41% reduction.

6.6 Ablation Study

We evaluate the contributions of per-request DCP and its data-plane support on the DeepSeek-V3 Issue 1% workload. Figure 16 reports mean and P99 TPOT under different request rates.

Effectiveness of DCP. As the request rate increases, NanoCP maintains stable latency, whereas uniform CP8 degrades rapidly. At 40 requests/s, replacing DCP with uniform CP8 increases mean/P99 TPOT from 49.08/52.07 ms to 109.97/126.77 ms. This is because uniform CP8 imposes unnecessary communication and reduces attention-kernel efficiency by inflating the effective attention batch on short requests.

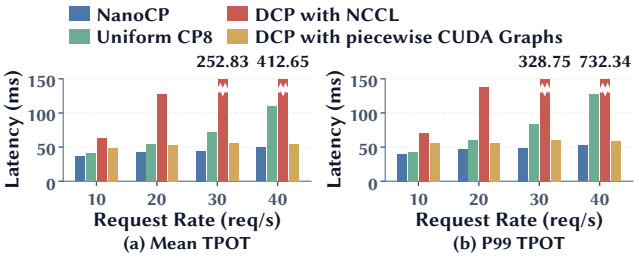


Figure 16. Ablation results on the DeepSeek-V3 Issue 1% workload under different request rates.

Impact of data-plane support. Replacing the routing backend with NCCL increases mean/P99 TPOT to 412.65/732.34 ms at 40 requests/s, as dense collectives poorly match DCP's sparse and asymmetric transfers. Piecewise CUDA Graphs achieve 54.80/57.88 ms, which are 11.7%/11.2% higher than NanoCP, demonstrating the benefit of capturing complete execution graphs specialized by the two-dimensional execution shape (M, N).

6.7 System Overhead Analysis

Scheduler and AOT graph engine. We measure the centralized control-plane overhead on 32 GPUs with per-instance batch sizes from 32 to 256 (Figure 17). Under 32DP, scheduling plus RDMA metadata transfer accounts for only 1.17%–1.77% of iteration time. Enabling DCP raises this to 1.42%–2.61%, an increase of at most 0.84 percentage points. Graph selection is implemented as a simple dictionary lookup over prebuilt graphs indexed by execution-shape buckets, taking less than 2μs per iteration. After selecting the graph, NanoCP performs one metadata injection per decode iteration. Under representative 4DP8CP settings from our end-to-end experiments, with per-GPU batch sizes up to 256 and Issue-1% sequence lengths, metadata injection takes approximately 20μs per iteration. The AOT graph engine prebuilds 48 CUDA Graphs consuming 5.32 GiB per GPU (Table 2), an additional 4.30 GiB (~3.0% of H200 HBM) over vLLM 32DP, comparable to vLLM 4DP8CP (5.17 GiB, 68 graphs).

Communication backend. Using DeepSeek-V3's query tensor (128 heads, head dimension 576, varying batch size), NanoCP's routing-based backend accelerates query all-to-all transfers by 1.05×–2.03× over NCCL across the evaluated batch sizes (Figure 18).

DCP cost at runtime. As shown in Figure 19, at peak only 25 of 2,279 requests (1.1%) use cross-instance CP, with DCP all-to-all latency peaking at just 35.5 μs.

7 Discussion

Interaction with operation-level overlap.

NanoFlow [80] and Two-Batch Overlap (TBO) [13] overlap computation and communication across sub-batches but leave request placement unchanged. Variable-length workloads can therefore create instance-level stragglers that shorten the overlap window. NanoCP instead mitigates this imbalance through dynamic KV cache placement and dual-balanced scheduling.

The benefit of TBO depends on the workload. Figure 20 shows that on ShareGPT-4o at 100 requests/s, it reduces mean TPOT from 82.33 to 56.70 ms and P99 TPOT from 93.61 to 67.67 ms. On Issue 1% at 10 requests/s, however, it increases mean TPOT from 37.99 to 64.49 ms and P99 TPOT from 51.93 to 92.75 ms, as long-context stragglers limit overlap while microbatching introduces additional overhead.

TBO remains compatible with NanoCP. Following DeepSeek's TBO pipeline [13], we can insert Q-Route and Res/LSE-Route

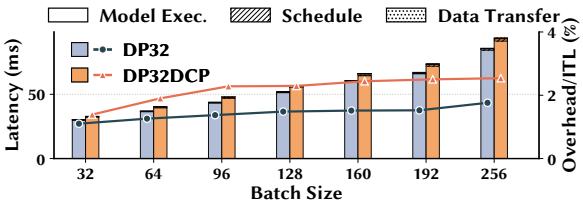


Figure 17. Schedule overhead comparison.

Table 2. CUDA Graph memory overhead comparison.

Method	# Graphs	Memory per GPU (GiB)
vLLM (4DP8CP)	68	5.17
vLLM (32DP)	12	1.02
NanoCP	48	5.32

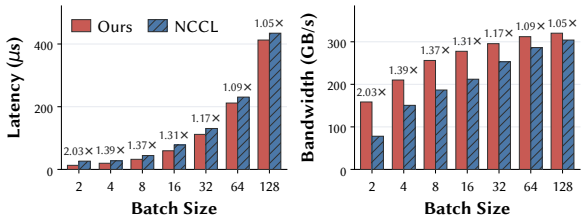


Figure 18. Performance of NanoCP's routing-based backend vs. NCCL for query all-to-all transfer. Labels show speedup over NCCL.

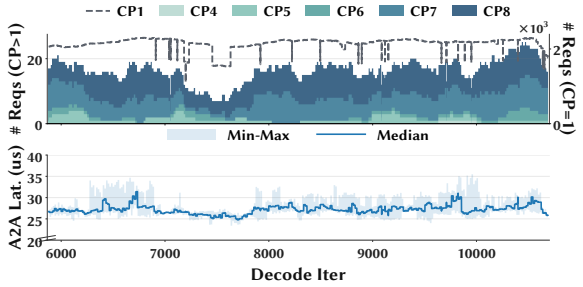


Figure 19. Runtime CP degrees and DCP communication latency.

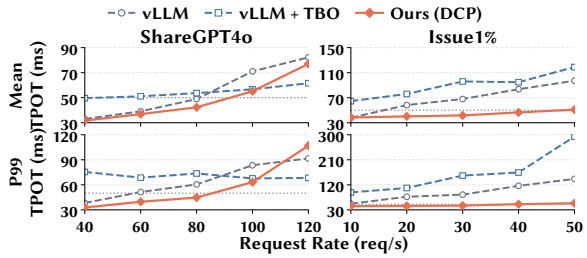


Figure 20. End-to-end performance with TBO.

around the attention stage, yielding *(SHARED + ATTN-0, Dispatch) → MLP → (Q-Route + ATTN-1 + Res/LSE-Route, Combine)*. The two techniques optimize placement and overlap independently; joint optimization is left to future work.

Generality and deployment scope. Under DP-EP serving, MoE models with MLA, GQA, or MHA can all suffer from imbalance in attention computation and MoE communication. Increasing TP does not eliminate this imbalance. For GQA models with few KV heads, scaling TP beyond the number of KV heads replicates KV heads and their caches; for MLA, head-wise TP may replicate the shared latent KV cache across TP ranks [4, 42]. Moreover, a moderate TP degree still leaves multiple DP instances with replicated attention layers, whose KV cache loads and batch sizes can diverge. SuperNode-class systems can accommodate more such instances within a high-bandwidth domain, allowing DCP to balance attention computation and MoE communication across more instances while pooling more KV cache capacity.

Our implementation confines each request's KV binding to one node. On our H200 cluster, intra-node placement already achieves effective load balance while avoiding Q/Res/LSE traffic on the inter-node critical path. The scheduling abstraction and routing design also support cross-node KV bindings, whose evaluation on SuperNode-class systems is left to future work.

Scalability. For larger deployments, NanoCP can adopt a hierarchical design: a lightweight global router assigns requests to GPU groups using aggregate load summaries, while each group's scheduler performs fine-grained DCP placement locally. We leave large-scale evaluation of this design to future work. To assess control-plane scalability, we use a scheduler-only CPU model to isolate the admission critical path across 4 and 32 logical nodes; each centralized/decentralized pair processes an identical request cohort. At four nodes, decentralized scheduling is comparable to centralized scheduling for Issue-1%/R90 (5.5 versus 5.0 ms) and is 1.36× faster for Issue-5%/R45 (18.0 versus 24.4 ms). At 32 logical nodes, it reduces admission time from 119.5 to 13.6 ms (8.80×) and from 109.3 to 19.0 ms (5.76×), respectively. This widening gap indicates that sharding scheduling state across local schedulers mitigates the centralized admission bottleneck as the aggregate scheduling state grows.

8 Related Work

Context parallelism for LLM serving. Context parallelism (CP) was developed to scale sequence length in training [5, 37, 53] and later adopted for serving. LoongServe [63], InfiniteLLM [36], Meta's context-parallel inference design [67], Medha [2], and Helix [4] use CP to pool distributed memory for long contexts or mitigate head-of-line blocking. LoongServe elastically adjusts CP at the group level, while Helix applies a uniform CP degree across requests. Neither independently selects each request's MoE and KV bindings within the same decoding step, preventing joint balancing of attention load and MoE communication. NanoCP instead uses per-request dynamic CP to decouple KV cache placement from MoE execution, achieving dual balance while avoiding unnecessary CP overhead for short requests.

MoE serving systems. Prior work optimizes MoE inference via efficient kernels and communication [16, 21, 35, 41, 73] or expert-level load balance [15, 38, 56, 70]. MegaScale-Infer [81] disaggregates attention and FFN but assumes balanced execution times. These optimizations are orthogonal to NanoCP, which targets the DP-instance-level straggler effect under lock-step EP synchronization.

Request scheduling in LLM serving. Existing systems [1, 31, 39, 40, 68, 77] bind each request's attention computation, MoE communication, and KV cache to a single DP instance. Llumnix [55] adds cross-instance migration, while DistServe [79], Splitwise [48], and Mooncake [51] disaggregate prefill and decode. SBS [58] further performs load balancing for both prefill and decode, but its decode scheduler still assigns each request to a single DP instance based on batch size and KV cache load. These designs therefore retain a single request-to-instance binding and do not decouple a request's MoE binding from its KV binding within a decoding step. NanoCP instead schedules the two bindings independently through dual-balanced scheduling with per-request CP degrees.

9 Conclusion

NanoCP addresses MoE inference conflicts by decoupling scheduling via dynamic KV cache placement and dual-balanced scheduling. Under the TPOT SLO, it improves maximum request rates by up to 3.27× and significantly improves P99 TPOT over existing strategies.

Acknowledgments

We thank the anonymous reviewers and our shepherd for their feedback, and Zihan Wang, Rongxin Cheng, and Qidong Su for insightful discussions and suggestions. This work was supported by the National Science and Technology Major Project for New Generation Artificial Intelligence (Project No. 2025ZD0123805).

References

[1] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, Santa Clara, CA, 117–134. https://www.usenix.org/conference/osdi24/presentation/agrawal

[2] Amey Agrawal, Haoran Qiu, Junda Chen, Íñigo Goiri, Chaojie Zhang, Rayyan Shahid, Ramachandran Ramjee, Alexey Tumanov, and Esha Choukse. 2025. No Request Left Behind: Tackling Heterogeneity in Long-Context LLM Inference with Medha. Accepted to ACM Symposium on Cloud Computing (SoCC '26). arXiv:2409.17264 [cs.LG] https://arxiv.org/abs/2409.17264

[3] Malika Aubakirova, Alex Atallah, Chris Clark, Justin Summerville, and Anjney Midha. 2026. State of AI: An Empirical 100 Trillion Token Study with OpenRouter. arXiv:2601.10088 doi:10.48550/arXiv.2601.10088

[4] Nidhi Bhatia, Ankit More, Ritika Borkar, Tiyasa Mitra, Ramon Matas, Ritchie Zhao, Maximilian Golub, Dheevatsa Mudigere, Brian Pharris, and Bita Darvish Rouhani. 2025. Helix Parallelism: Rethinking Sharding Strategies for Interactive Multi-Million-Token LLM Decoding. arXiv:2507.07120 doi:10.48550/arXiv.2507.07120

[5] Chang Chen, Tiancheng Chen, Jiangfei Duan, Qianchao Zhu, Zerui Wang, Qinghao Hu, Peng Sun, Xiuhong Li, Chao Yang, and Torsten Hoefler. 2026. Zeppelin: Balancing Variable-length Workloads in Data Parallel Large Model Training. In *Proceedings of the 21st European Conference on Computer Systems* (McEwan Hall/The University of Edinburgh, Edinburgh, Scotland UK) *(EUROSYS '26)*. Association for Computing Machinery, New York, NY, USA, 1879–1893. doi:10.1145/3767295.3769369

[6] Chang Chen, Min Li, Zhihua Wu, Dianhai Yu, and Chao Yang. 2022. TA-MoE: topology-aware large scale mixture-of-expert training. In *Proceedings of the 36th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) *(NIPS '22)*. Curran Associates Inc., Red Hook, NY, USA, Article 1611, 14 pages. doi:10.52202/068431-1611

[7] Hongtao Chen, Weiyu Xie, Boxin Zhang, Jingqi Tang, Jiahao Wang, Jianwei Dong, Shaoyuan Chen, Ziwei Yuan, Chen Lin, Chengyu Qiu, Yuening Zhu, Qingliang Ou, Jiaqi Liao, Xianglin Chen, Zhiyuan Ai, Yongwei Wu, and Mingxing Zhang. 2025. KTransformers: Unleashing the Full Potential of CPU/GPU Hybrid Inference for MoE Models. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles* (Lotte Hotel World, Seoul, Republic of Korea) *(SOSP '25)*. Association for Computing Machinery, New York, NY, USA, 1014–1029. doi:10.1145/3731569.3764843

[8] Meghan Cowan, Saeed Maleki, Madanlal Musuvathi, Olli Saarikivi, and Yifan Xiong. 2023. MSCCLang: Microsoft Collective Communication Language. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (Vancouver, BC, Canada) *(ASPLOS 2023)*. ACM, New York, NY, USA, 502–514. doi:10.1145/3575693.3575724

[9] Erfei Cui, Yinan He, Zheng Ma, Zhe Chen, Hao Tian, Weiyun Wang, Kunchang Li, Yi Wang, Wenhai Wang, Xizhou Zhu, Lewei Lu, Tong Lu, Yali Wang, Limin Wang, Yu Qiao, and Jifeng Dai. 2024. ShareGPT-4o: Comprehensive Multimodal Annotations with GPT-4o. https://sharegpt4o.github.io/

[10] Tri Dao, Daniel Haziza, Francisco Massa, and Grigory Sizov. 2023. Flash-Decoding for long-context inference. PyTorch Blog. https://pytorch.org/blog/flash-decoding/

[11] DeepSeek-AI. 2024. DeepSeek-V3 Technical Report. arXiv:2412.19437 [cs.CL] https://arxiv.org/abs/2412.19437

[12] DeepSeek-AI. 2025. EPLB: Expert Parallelism Load Balancer. GitHub repository. Retrieved August 26, 2026 from https://github.com/deepseek-ai/EPLB

[13] DeepSeek-AI. 2025. Profiling Data in DeepSeek Infra. GitHub repository. Commit `4496024`. Retrieved August 26, 2026 from https://github.com/deepseek-ai/profile-data/tree/4496024

[14] Xiang Deng, Jeff Da, Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Nitin Pasari, Chetan Rane, Karmini Sampath, Maya Krishnan, Srivatsa Kundurthy, Sean Hendryx, Zifan Wang, Vijay Bharadwaj, Jeff Holm, Raja Aluri, Chen Bo Calvin Zhang, Noah Jacobson, Bing Liu, and Brad Kenstler. 2025. SWE-Bench Pro: Can AI Agents Solve Long-Horizon Software Engineering Tasks? arXiv:2509.16941 [cs.SE] https://arxiv.org/abs/2509.16941

[15] William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research* 23, 120 (2022), 1–39. https://jmlr.org/papers/v23/21-0998.html

[16] Trevor Gale, Deepak Narayanan, Cliff Young, and Matei Zaharia. 2023. MegaBlocks: Efficient Sparse Training with Mixture-of-Experts. In *Proceedings of Machine Learning and Systems*, D. Song, M. Carbin, and T. Chen (Eds.), Vol. 5. 288–304. https://proceedings.mlsys.org/paper_files/paper/2023/file/5a54f79333768effe7e8927bcccffe40-Paper-mlsys2023.pdf

[17] Hao Ge, Fangcheng Fu, Haoyang Li, Xuanyu Wang, Sheng Lin, Yujie Wang, Xiaonan Nie, Hailin Zhang, Xupeng Miao, and Bin Cui. 2024. Enabling Parallelism Hot Switching for Efficient Training of Large Language Models. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles* (Austin, TX, USA) *(SOSP '24)*. Association for Computing Machinery, New York, NY, USA, 178–194. doi:10.1145/3694715.3695969

[18] Gemini Team, Google. 2024. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. arXiv:2403.05530 [cs.CL] https://arxiv.org/abs/2403.05530

[19] Abhishek Ghosh, Ajay Nayak, Ashish Panwar, and Arkaprava Basu. 2026. GraCE: Unlocking CUDA Graphs with Compiler Support for ML Workloads. In *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26)*. USENIX Association, Seattle, WA, 1927–1947. https://www.usenix.org/conference/osdi26/presentation/ghosh

[20] Seokjin Go and Divya Mahajan. 2025. MoETuner: Optimized Mixture of Expert Serving with Balanced Expert Placement and Token Routing. arXiv:2502.06643 [cs.LG] https://arxiv.org/abs/2502.06643

[21] Amos Goldman, Nimrod Boker, Maayan Sheraizin, Nimrod Admoni, Artem Polyakov, Subhadeep Bhattacharya, Fan Yu, Kai Sun, Georgios Theodorakis, Hsin-Chun Yin, Peter-Jan Gootzen, Aamir Shafi, Assaf Ravid, Salvatore Di Girolamo, James Dinan, Xiaofan Li, Manjunath Gorentla Venkata, and Gil Bloch. 2026. NCCL EP: Towards a Unified Expert Parallel Communication API for NCCL. arXiv:2603.13606 [cs.DC] https://arxiv.org/abs/2603.13606

[22] Shwai He, Weilin Cai, Jiayi Huang, and Ang Li. 2026. Capacity-Aware Inference: Mitigating the Straggler Effect in Mixture of Experts. In *The Fourteenth International Conference on Learning Representations*. https://openreview.net/forum?id=LuYFpySWA2

[23] Junhao Hu, Jiang Xu, Zhixia Liu, Yulong He, Yuetao Chen, Hao Xu, Jiang Liu, Jie Meng, Baoquan Zhang, Shining Wan, Gengyuan Dan, Zhiyu Dong, Zhihao Ren, Changhong Liu, Tao Xie, Dayun Lin, Qin Zhang, Yue Yu, Hao Feng, Xusheng Chen, and Yizhou Shan. 2025. DEEPSERVE: Serverless Large Language Model Serving at Scale. In *2025 USENIX Annual Technical Conference (USENIX ATC 25)*. USENIX Association, Boston, MA, 57–72. https://www.usenix.org/conference/atc25/presentation/hu-junhao

[24] Changho Hwang, Peng Cheng, Roshan Dathathri, Abhinav Jangda, Saeed Maleki, Madan Musuvathi, Olli Saarikivi, Aashaka Shah, Ziyue Yang, Binyang Li, Caio Rocha, Qinghua Zhou, Mahdieh Ghazimirsaeed, Sreevatsa Anantharamu, and Jithin Jose. 2026. MSCCL++: Rethinking GPU Communication Abstractions for AI Inference. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (USA) *(ASPLOS '26)*. ACM, New York, NY, USA, 1201–1215.

doi:10.1145/3779212.3790188

[25] Sam Ade Jacobs, Masahiro Tanaka, Chengming Zhang, Minjia Zhang, Reza Yazdani Aminadabi, Shuaiwen Leon Song, Samyam Rajbhandari, and Yuxiong He. 2024. System optimizations for enabling training of extreme long sequence transformer models. In *Proceedings of the 43rd ACM Symposium on Principles of Distributed Computing*. 121–130. doi:10.1145/3662158.3662806

[26] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, Gianna Lengyel, Guillaume Bour, Guillaume Lample, Lélio Renard Lavaud, Lucile Saulnier, Marie-Anne Lachaux, Pierre Stock, Sandeep Subramanian, Sophia Yang, Szymon Antoniak, Teven Le Scao, Théophile Gervet, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2024. Mixtral of Experts. arXiv:2401.04088 [cs.LG] https://arxiv.org/abs/2401.04088

[27] Chenyu Jiang, Zhenkun Cai, Ye Tian, Zhen Jia, Yida Wang, and Chuan Wu. 2025. DCP: Addressing Input Dynamism In Long-Context Training via Dynamic Context Parallelism. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*. 221–236. doi:10.1145/3731569.3764849

[28] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-world Github Issues?. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net. https://proceedings.iclr.cc/paper_files/paper/2024/hash/edac78c3e300629acfe6cbe9ca88fb84-Abstract-Conference.html

[29] Kimi Team. 2025. Kimi K2: Open Agentic Intelligence. arXiv:2507.20534 [cs.LG] https://arxiv.org/abs/2507.20534

[30] Kimi Team. 2026. Kimi K2.5: Visual Agentic Intelligence. arXiv:2602.02276 [cs.CL] https://arxiv.org/abs/2602.02276

[31] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles* (Koblenz, Germany) *(SOSP '23)*. Association for Computing Machinery, New York, NY, USA, 611–626. doi:10.1145/3600006.3613165

[32] Cong Li, Yuzhe Yang, Xuegui Zheng, Qifan Yang, Yijin Guan, Size Zheng, Li-Wen Chang, Shufan Liu, Xin Liu, and Guangyu Sun. 2026. Tetris: Efficient Long-context LLM Serving with Chunkwise Dynamic Sequence Parallelism. In *2026 ACM/IEEE 53rd Annual International Symposium on Computer Architecture (ISCA)*. 1287–1301. doi:10.1109/ISCA66397.2026.00098

[33] Jiashi Li and Shengyu Liu. 2025. FlashMLA: Efficient Multi-head Latent Attention Kernels. GitHub repository. Retrieved August 26, 2026 from https://github.com/deepseek-ai/FlashMLA

[34] Yan Li, Zhenyu Zhang, Zhengang Wang, Pengfei Chen, and Pengfei Zheng. 2026. Semantic Parallelism: Redefining Efficient MoE Inference via Model-Data Co-Scheduling. In *International Conference on Learning Representations*, C. Vondrick, B. Hariharan, C. Raffel, L. Pinto, D. Yang, and A. Faust (Eds.), Vol. 2026. 148580–148599. https://proceedings.iclr.cc/paper_files/paper/2026/file/f0552f14388d95b19740dee809f5cad1-Paper-Conference.pdf

[35] Nandor Licker, Kevin Hu, Vladimir Zaytsev, and Lequn Chen. 2026. fabric-lib: RDMA Point-to-Point Communication for LLM Systems. In *Ninth Conference on Machine Learning and Systems*. https://openreview.net/forum?id=SjVa05wEiY

[36] Bin Lin, Chen Zhang, Tao Peng, Hanyu Zhao, Wencong Xiao, Minmin Sun, Anmin Liu, Zhipeng Zhang, Lanbo Li, Xiafei Qiu, Shen Li, Zhigang Ji, Tao Xie, Yong Li, and Wei Lin. 2024. Infinite-LLM: Efficient LLM Service for Long Context with DistAttention and Distributed KVCache. arXiv:2401.02669 [cs.DC] https://arxiv.org/abs/2401.02669

[37] Hao Liu, Matei Zaharia, and Pieter Abbeel. 2024. RingAttention with Blockwise Transformers for Near-Infinite Context. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net. https://proceedings.iclr.cc/paper_files/paper/2024/hash/1119587863e78451f080da2a768c4935-Abstract-Conference.html

[38] Ziming Liu, Boyu Tian, Guoteng Wang, Zhen Jiang, Peng Sun, Zhenhua Han, Tian Tang, Xiaohe Hu, Yanmin Jia, Yan Zhang, He Liu, Mingjun Zhang, Yiqi Zhang, Qiaoling Chen, Shenggan Cheng, Mingyu Gao, Yang You, and Siyuan Feng. 2025. Expert-as-a-Service: Towards Efficient, Scalable, and Robust Large-scale MoE Serving. arXiv:2509.17863 [cs.DC] https://arxiv.org/abs/2509.17863

[39] LMDeploy Contributors. 2023. LMDeploy: A Toolkit for Compressing, Deploying, and Serving LLMs. GitHub repository. Retrieved August 26, 2026 from https://github.com/InternLM/lmdeploy

[40] Jinming Ma, Jiefei Chen, Xiuhong Li, Jiangfei Duan, Haojie Duanmu, Xingcheng Zhang, Chao Yang, and Dahua Lin. 2025. Tropical: Enhancing SLO Attainment in Disaggregated LLM Serving via SLO-Aware Multiplexing. In *2025 62nd ACM/IEEE Design Automation Conference (DAC)*. 1–7. doi:10.1109/DAC63849.2025.11132617

[41] Ziming Mao, Yihan Zhang, Chihan Cui, Zhen Huang, Kaichao You, Zhongjie Chen, Zhiying Xu, Zhenyu Gu, Scott Shenker, Costin Raiciu, Yang Zhou, and Ion Stoica. 2026. UEP: Portable Expert-Parallel Communication. In *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26)*. USENIX Association, Seattle, WA, 1107–1123. https://www.usenix.org/conference/osdi26/presentation/mao-ziming-uep

[42] Tiyasa Mitra, Ritika Borkar, Nidhi Bhatia, Shivam Raj, Hongkuan Zhou, Yan Ru Pei, Vishwanath Venkatesan, Kyle Kranen, Ramon Matas, Dheevatsa Mudigere, Ritchie Zhao, Maximilian Golub, Arpan Dutta, Suresh Nambi, Sailaja Madduri, Dharmesh Jani, Brian Pharris, Itay Neeman, and Bita Darvish Rouhani. 2026. Beyond the Buzz: A Pragmatic Take on Inference Disaggregation. In *Proceedings of Machine Learning and Systems*, A. Chowdhery and Z. Jia (Eds.), Vol. 8. MLSys, 2001–2015. https://proceedings.mlsys.org/paper_files/paper/2026/file/d49cee5f3a79d97d719df255689d83d7-Paper-Conference.pdf

[43] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I. Jordan, and Ion Stoica. 2018. Ray: A Distributed Framework for Emerging AI Applications. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. USENIX Association, Carlsbad, CA, 561–577. https://www.usenix.org/conference/osdi18/presentation/moritz

[44] NVIDIA Corporation. 2026. NCCL: Optimized Primitives for Inter-GPU Communication. GitHub repository. Retrieved August 26, 2026 from https://github.com/NVIDIA/nccl

[45] Gabriele Oliaro, Xupeng Miao, Xinhao Cheng, Vineeth Kada, Mengdi Wu, Ruohan Gao, Yingyi Huang, Remi Delacourt, April Yang, Yingcheng Wang, Colin Unger, and Zhihao Jia. 2026. FlexLLM: Token-Level Co-Serving of LLM Inference and Finetuning with SLO Guarantees. In *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*. USENIX Association, Renton, WA, 1359–1379. https://www.usenix.org/conference/nsdi26/presentation/oliaro

[46] OpenAI. 2026. *Introducing GPT-5.4*. OpenAI. Retrieved April 16, 2026 from https://openai.com/index/introducing-gpt-5-4/

[47] OpenRouter. 2026. *OpenRouter Rankings: Model Usage Statistics*. Retrieved February 23, 2026 from https://openrouter.ai/rankings

[48] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. 118–132. doi:10.1109/ISCA59077.2024.00019

[49] Sundar Pichai, Demis Hassabis, and Koray Kavukcuoglu. 2025. *A New Era of Intelligence with Gemini 3*. Google. Retrieved April 16, 2026 from https://blog.google/products-and-platforms/products/gemini/gemini-3/

[50] PyTorch Foundation. 2023. Accelerating Generative AI with PyTorch II: GPT, Fast. https://pytorch.org/blog/accelerating-generative-ai-2/. Accessed: 2026-04-16.

[51] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2025. Mooncake: Trading More Storage for Less Computation — A KVCache-centric Architecture for Serving LLM Chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*. USENIX Association, Santa Clara, CA, 155–170. https://www.usenix.org/conference/fast25/presentation/qin

[52] Qwen Team. 2026. Qwen3.5: Towards Native Multimodal Agents. https://qwen.ai/blog?id=qwen3.5

[53] Samyam Rajbhandari, Conglong Li, Zhewei Yao, Minjia Zhang, Reza Yazdani Aminabadi, Ammar Ahmad Awan, Jeff Rasley, and Yuxiong He. 2022. DeepSpeed-MoE: Advancing Mixture-of-Experts Inference and Training to Power Next-Generation AI Scale. In *International Conference on Machine Learning, ICML 2022, 17-23 July 2022, Baltimore, Maryland, USA (Proceedings of Machine Learning Research, Vol. 162)*. PMLR, 18332–18346. https://proceedings.mlr.press/v162/rajbhandari22a.html

[54] StepFun Team. 2025. Step-3 is Large yet Affordable: Model-system Co-design for Cost-effective Decoding. arXiv:2507.19427 [cs.LG] https://arxiv.org/abs/2507.19427

[55] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. 2024. Llumnix: Dynamic Scheduling for Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, Santa Clara, CA, 173–191. https://www.usenix.org/conference/osdi24/presentation/sun-biao

[56] Xinru Tang, Jingxiang Hou, Dingcheng Jiang, Taiquan Wei, Jiaxin Liu, Jinyi Deng, Huizheng Wang, Qize Yang, Haoran Shang, Chao Li, Yang Hu, and Shouyi Yin. 2026. MoEntwine: Unleashing the Potential of Wafer-Scale Chips for Large-Scale Expert Parallel Inference . In *2026 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE Computer Society, Los Alamitos, CA, USA, 1–15. doi:10.1109/HPCA68181.2026.11408594

[57] Xiaojuan Tang, Fanxu Meng, Pingzhi Tang, Yuxuan Wang, Di Yin, Xing Sun, and Muhan Zhang. 2026. TPLA: Tensor Parallel Latent Attention for Efficient Disaggregated Prefill & Decode Inference. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (USA) *(ASPLOS '26)*. Association for Computing Machinery, New York, NY, USA, 2048–2062. doi:10.1145/3779212.3790237

[58] Jian Tian, Shuailong Li, Yang Cao, Wenbo Cui, Minghan Zhu, Wenkang Wu, Jianming Zhang, Yanpeng Wang, Zhiwen Xiao, Zhenyu Hou, and Dou Shen. 2025. Staggered Batch Scheduling: Co-optimizing Time-to-First-Token and Throughput for High-Efficiency LLM Inference. arXiv:2512.16134 [cs.DC] https://arxiv.org/abs/2512.16134

[59] Ilyas Turimbetov, Mohamed Wahib, and Didem Unat. 2025. A Device-Side Execution Model for Multi-GPU Task Graphs. In *Proceedings of the 39th ACM International Conference on Supercomputing (ICS '25)*. Association for Computing Machinery, New York, NY, USA, 384–396. doi:10.1145/3721145.3730426

[60] Hulin Wang, Yaqi Xia, Donglin Yang, Xiaobo Zhou, and Dazhao Cheng. 2025. Harnessing Inter-GPU Shared Memory for Seamless MoE Communication-Computation Fusion. In *Proceedings of the 30th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming* (Las Vegas, NV, USA) *(PPoPP '25)*. Association for Computing Machinery, New York, NY, USA, 170–182. doi:10.1145/3710848.3710868

[61] Ne Wang, Wenxiang Lin, Lin Zhang, Shaohuai Shi, Ruiting Zhou, and Bo Li. 2025. SP-MoE: Expediting Mixture-of-Experts Training with Optimized Pipelining Planning. In *IEEE INFOCOM 2025 - IEEE Conference on Computer Communications*. 1–10. doi:10.1109/INFOCOM55648.2025.11044508

[62] Yujie Wang, Shiju Wang, Shenhan Zhu, Fangcheng Fu, Xinyi Liu, Xuefeng Xiao, Huixia Li, Jiashi Li, Faming Wu, and Bin Cui. 2025. FlexSP: Accelerating Large Language Model Training via Flexible Sequence Parallelism. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (Rotterdam, Netherlands) *(ASPLOS '25)*. Association for Computing Machinery, New York, NY, USA, 421–436. doi:10.1145/3676641.3715998

[63] Bingyang Wu, Shengyu Liu, Yinmin Zhong, Peng Sun, Xuanzhe Liu, and Xin Jin. 2024. Loongserve: Efficiently serving long-context large language models with elastic sequence parallelism. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*. 640–654. doi:10.1145/3694715.3695948

[64] Bingyang Wu, Zili Zhang, Yinmin Zhong, Guanzhe Huang, Yibo Zhu, Xuanzhe Liu, and Xin Jin. 2025. TokenLake: A Unified Segment-level Prefix Cache Pool for Fine-grained Elastic Long-Context LLM Serving. arXiv:2508.17219 [cs.DC] https://arxiv.org/abs/2508.17219

[65] Yueru Yan, Tuc Nguyen, Bo Su, Melissa Lieffers, and Thai Le. 2026. ShareChat: A Dataset of Chatbot Conversations in the Wild. arXiv:2512.17843 [cs.CL] https://arxiv.org/abs/2512.17843

[66] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. 2025. Qwen3 Technical Report. arXiv:2505.09388 [cs.CL] https://arxiv.org/abs/2505.09388

[67] Amy Yang, Jingyi Yang, Aya Ibrahim, Xinfeng Xie, Bangsheng Tang, Grigory Sizov, Jongsoo Park, and Jianyu Huang. 2025. Context Parallelism for Scalable Million-Token Inference. In *Proceedings of Machine Learning and Systems*, M. Zaharia, G. Joshi, and Y. Lin (Eds.), Vol. 7. MLSys. https://proceedings.mlsys.org/paper_files/paper/2025/file/78834433edc3291f4c6cbbd2759324db-Paper-Conference.pdf

[68] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. USENIX Association, Carlsbad, CA, 521–538. https://www.usenix.org/conference/osdi22/presentation/yu

[69] Xingkai Yu. 2025. Nano-vLLM: A Lightweight vLLM Implementation Built from Scratch. GitHub repository. Version 0.2.0. Retrieved August 26, 2026 from https://github.com/GeeeekExplorer/nano-vllm

[70] Zhexiang Zhang, Ye Wang, Yumiao Zhao, Jiayu Xiao, Qianjing Yang, Xiangyu Wang, Jingzhe Jiang, Qizhen Weng, Ruichuan Chen, Shaohuai Shi, Yin Chen, and Minchen Yu. 2026. Janus: Disaggregating Attention and Experts for Scalable MoE Inference. arXiv:2512.13525 [cs.DC] https://arxiv.org/abs/2512.13525

[71] Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Huazuo Gao, Jiashi Li, Liyue Zhang, Panpan Huang, Shangyan Zhou, Shirong Ma, Wenfeng Liang, Ying He, Yuqing Wang, Yuxuan Liu, and Y.X. Wei. 2025. Insights into DeepSeek-V3: Scaling Challenges and Reflections on Hardware for AI Architectures. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA '25)*. Association for Computing Machinery, New York, NY, USA, 1731–1745. doi:10.1145/3695053.3731412

[72] Chenggang Zhao, Zhean Xu, Liang Zhao, Jiashi Li, Chenhao Xu, Anyi Xu, Shengyu Liu, Kexing Zhou, and Kuai Yu. 2025. DeepGEMM:

clean and efficient BLAS kernel library on GPU. GitHub repository. Retrieved August 26, 2026 from https://github.com/deepseek-ai/DeepGEMM

[73] Chenggang Zhao, Shangyan Zhou, Liyue Zhang, Chengqi Deng, Zhean Xu, Yuxuan Liu, Kuai Yu, Jiashi Li, and Liang Zhao. 2025. DeepEP: an efficient expert-parallel communication library. GitHub repository. Retrieved August 26, 2026 from https://github.com/deepseek-ai/DeepEP

[74] Wenting Zhao, Xiang Ren, Jack Hessel, Claire Cardie, Yejin Choi, and Yuntian Deng. 2024. WildChat: 1M ChatGPT Interaction Logs in the Wild. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net. https://proceedings.iclr.cc/paper_files/paper/2024/hash/9421261e06f1a63a352b068f1ac90609-Abstract-Conference.html

[75] Bojian Zheng, Cody Hao Yu, Jie Wang, Yaoyao Ding, Yizhi Liu, Yida Wang, and Gennady Pekhimenko. 2023. Grape: Practical and Efficient Graphed Execution for Dynamic Deep Neural Networks on GPUs. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture* (Toronto, ON, Canada) *(MICRO '23)*. Association for Computing Machinery, New York, NY, USA, 1364–1380. doi:10.1145/3613424.3614248

[76] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Tianle Li, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zhuohan Li, Zi Lin, Eric P. Xing, Joseph E. Gonzalez, Ion Stoica, and Hao Zhang. 2024. LMSYS-Chat-1M: A Large-Scale Real-World LLM Conversation Dataset. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net. https://proceedings.iclr.cc/paper_files/paper/2024/hash/5f9bfdfe3685e4ccdbc0e7fb29cccf2a-Abstract-Conference.html

[77] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2024. SGLang: Efficient Execution of Structured Language Model Programs. In *Advances in Neural Information Processing Systems*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (Eds.), Vol. 37. Curran Associates, Inc., 62557–62583. doi:10.52202/079017-2000

[78] Size Zheng, Wenlei Bao, Qi Hou, Xuegui Zheng, Jin Fang, Chenhui Huang, Tianqi Li, Haojie Duanmu, Renze Chen, Ruifan Xu, Yifan Guo, Ningxin Zheng, Ziheng Jiang, Xinyi Di, Dongyang Wang, Jianxi Ye, Haibin Lin, Li-Wen Chang, Liqiang Lu, Yun Liang, Jidong Zhai, and Xin Liu. 2025. Triton-distributed: Programming Overlapping Kernels on Distributed AI Systems with the Triton Compiler. arXiv:2504.19442 [cs.DC] https://arxiv.org/abs/2504.19442

[79] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 193–210. https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin

[80] Kan Zhu, Yufei Gao, Yilong Zhao, Liangyu Zhao, Gefei Zuo, Yile Gu, Dedong Xie, Tian Tang, Qinyu Xu, Zihao Ye, Keisuke Kamahori, Chien-Yu Lin, Ziren Wang, Stephanie Wang, Arvind Krishnamurthy, and Baris Kasikci. 2025. NanoFlow: Towards Optimal Large Language Model Serving Throughput. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*. USENIX Association, Boston, MA, 749–765. https://www.usenix.org/conference/osdi25/presentation/zhu-kan

[81] Ruidong Zhu, Ziheng Jiang, Chao Jin, Peng Wu, Cesar A. Stuardo, Dongyang Wang, Xinlei Zhang, Huaping Zhou, Haoran Wei, Yang Cheng, Jianzhe Xiao, Xinyi Zhang, Lingjun Liu, Haibin Lin, Li-Wen Chang, Jianxi Ye, Xiao Yu, Xuanzhe Liu, Xin Jin, and Xin Liu. 2025. MegaScale-Infer: Efficient Mixture-of-Experts Model Serving with Disaggregated Expert Parallelism. In *Proceedings of the ACM SIGCOMM 2025 Conference* (São Francisco Convent, Coimbra, Portugal) *(SIGCOMM '25)*. Association for Computing Machinery, New York, NY, USA, 592–608. doi:10.1145/3718958.3750506

A Artifact Appendix

A.1 Abstract

The NanoCP artifact includes the system implementation, benchmark scripts, workload traces and utilities, and plot generators. It supports the evaluation in this paper and provides recorded H200 latencies for reconstructing the CP-degree selection policy in Algorithm 1. Offline policy reconstruction requires only Python 3; serving experiments use the software and hardware configurations described below.

A.2 Artifact check-list (meta-information)

- **Methods:** NanoCP (Python/C++), dynamic context parallelism, dual-balanced scheduling, offline policy search, and vLLM baselines.
- **Models:** DeepSeek-V3 and Kimi-K2-Instruct-0905.
- **Workloads:** ShareGPT-4o and GitHub Issues traces, including 1% and 5% long-context mixtures.
- **Runtime:** Linux; CUDA; PyTorch; Ray; NVLink and RDMA; modified vLLM; pinned DeepEP, DeepGEMM, FlashMLA, and DLSlime.
- **Hardware:** The full serving evaluation uses 32 GPUs across four nodes. See the figure-specific READMEs for component-test requirements.
- **Outputs:** mean/P99 time per output token (TPOT), 50-ms TPOT-SLO attainment, request rate, component latencies, logs, and generated figures.
- **License:** MIT for released source; third-party models and data retain their upstream licenses.

A.3 Access and dependencies

Public archive. The artifact is archived on Zenodo. Start with `ae_scripts/README.md`; each figure-specific README provides the commands, resource requirements, validation procedures, and expected outputs.

Models and environment. Model checkpoints must be obtained separately. The artifact provides a Dockerfile specifying the pinned software environment, including the modified vLLM baseline. Model and dataset paths are configured in `paths.env`, using the example under `ae_scripts/`. Distributed experiments use Linux with CUDA, GPU-aware RDMA, and identical absolute paths across workers. Storage requirements are dominated by model checkpoints and generated logs.

Data sets. The artifact includes four preprocessed request-length traces under `ae_scripts/dataset/`: ShareGPT-4o, GitHub Issues, and two mixed workloads containing 1% and 5% long-context requests, respectively.

A.4 Installation and basic testing

Fetch the archived source; run commands from `ae_scripts/`.

Evaluated environment. The experiments reported in the paper were conducted on four nodes, each equipped with eight NVIDIA H200 GPUs with 141 GB of memory, intra-node NVLink, and inter-node RDMA. The evaluated software environment is specified by the Dockerfile under `ae_scripts/docker/`.

Deployment on other clusters. Follow `ae_scripts/docker/README.md` to build the image and `ae_scripts/README.md` to create the containers and configure shared paths, the Ray cluster, worker SSH access, and RDMA settings. Copy `paths.env.example` to `paths.env` and configure the paths required by the selected experiments consistently across all nodes. Adjust the documented network addresses and RDMA settings to match the target cluster. The offline policy test below requires only Python 3 and runs without GPUs or the container.

Offline policy test (no GPUs required):

```
python3 algorithm/search_sp_buckets.py
```

Using the included grid of 258 sequence lengths and eight CP degrees, this script validates the data, reports the length-to-degree mapping, and saves the JSON policy to `algorithm/reproduced_policy.json`. GPU remeasurement is optional.

Distributed functional test (16 H200 GPUs): After configuring two nodes, run:

```
python3 basic-test/run_basic_test.py
```

Follow `basic-test/README.md` for the test procedure and expected results.

A.5 Experiments and expected results

Each figure README gives its measurement and plotting procedures. **Algorithm 1** uses `algorithm/` for offline policy reconstruction. **Figures 3, 5, and 6** use `fig3/`, `fig5/`, and `fig6/` for operator and imbalance measurements. **Figures 12–17 and 20** use the corresponding `fig*/` workflows for serving comparisons, profiling, and ablations on 32 GPUs; the complete Figure 12 sweep takes approximately four days. **Figures 18–19** measure collective communication on eight GPUs, with Figure 19 additionally requiring a four-node trace capture.

For DeepSeek-V3 at $\geq 99\%$ attainment of the 50-ms TPOT SLO, NanoCP sustains 3.27× and 2.11× the best evaluated vLLM request rates on Issue 1% and Issue 5%, respectively (Figure 12). Results may vary with hardware, software, and load.